Tiered Lottery Scheduler (TLSx64) Architecture Overview

The **Tiered Lottery Scheduler (TLSx64)** is an operating system CPU scheduler designed for $O(1)$ time complexity, deterministic fairness, low-latency interactivity, and symmetric multiprocessing (SMP) scalability on x86_64 architectures.

---

1. Tiers and the Ticket Pool

Instead of maintaining a single monolithic runqueue, TLSx64 distributes active processes across discrete priority buckets (Tiers). Every process assigned to a specific tier holds an equal, predetermined number of lottery tickets, establishing a proportional probability of selection.

Tier	Label	Tickets / Task	Purpose & Characteristics
Tier -1	**The Sanctuary**	N/A	Strict-priority lane for bottom-half hardware I/O routines. Completely bypasses the lottery, constrained by a strict hardware cycle budget (`RDTSC`) to prevent interrupt livelocks.
Tier 0	**The Anchor**	30	Fixed landing zone for newly spawned tasks and deferred I/O packages. Structurally excluded from starvation rotation.
Tier 1	**High Priority**	20	Fast, interactive tasks requiring minimal latency (e.g., UI updates, user input handlers).
Tier 2	**Medium Priority**	12	Standard interactive applications and intermediate computational tasks.
Tier 3	**Low Priority**	6	Heavy, throughput-oriented compute tasks.
Tier 4	**Background**	2	CPU-intensive background tasks and bulk background workers.

---

2. Rescheduling & $O(1)$ Winner Selection

Every 10 milliseconds, a local timer interrupt triggers a reschedule cycle on the local core:

1. **Calculate Total Ticket Pool:**
   The scheduler calculates the total active tickets by multiplying each tier's **active process count (kept and updated for every tier)** by its ticket weight and summing the values:

   $$\text{Total Tickets} = (T_0 \times 30) + (T_1 \times 20) + (T_2 \times 12) + (T_3 \times 6) + (T_4 \times 2)$$

   (Optimized by the compiler via reciprocal multiplication into a fixed, handful of clock cycles).
2. **Generate Winning Ticket:**
   To prevent pipeline stalls and division latency, the core couples hardware entropy (`RDRAND` / `RDSEED`) with a high-performance Xorshift PRNG. The output is scaled across the active ticket range via proportional fixed-point arithmetic rather than modulo division:

   $$\text{Winning_Ticket} = (\text{xorshift_value} \times \text{total_tickets}) \gg 32$$

   This ensures fast execution.
3. **Isolate Winner via Boundary Checks:**
   The scheduler performs sequential boundary checks from Tier 0 downward. When the range is identified, dividing the offset by that tier's ticket weight yields the exact index of the winning process in memory. Compilers turn this division into an instantaneous reciprocal multiply, maintaining strict $O(1)$ execution.

---

3. Demotion & Starvation Prevention

- **Swap-and-Pop Demotion:**
  When a process exhausts its assigned time quantum, it is demoted to the next lower tier. Rather than shifting array elements, the scheduler replaces the demoted process's slot with the last element in the array and appends the demoted task to the lower tier—executing in strict $O(1)$ operations.
- **Starvation Rotation:**
  Every 150 milliseconds, an independent hardware timer executes an $O(1)$ **pointer** rotation among general-purpose tiers: Tier 4 is promoted to Tier 1, while Tiers 1–3 shift downward by one level. Tiers -1 and 0 are locked, preventing priority inversion on core interrupt and I/O pipelines.

---

4. Symmetric Multiprocessing (SMP) & Cache Locality

To scale efficiently across multiple CPU cores without global lock contention:

- **Per-Core Runqueues:**
  Each physical core maintains its own independent tier arrays, ticket counter, PRNG state, and dedicated spinlock.
- **Passive Load Placement (Initial Drop):**
  New tasks are routed locklessly to the core with the lowest atomic **Load Factor (calculated from active tasks in Tier -1, Tier 3, and Tier 4)**.
- **Bottom-Up Batch Stealing:**
  Every 100 milliseconds, a core checks for load imbalances across neighboring cores. If an imbalance exceeds threshold parameters, it acquires the target core's spinlock and **pointer** swaps the lowest Tier with it's Tier 2.
- **Hardware Cache Preservation:**
  Work-stealing strictly targets Tier 4 (and cascades to Tier 3 only if necessary). High-priority interactive tasks in Tiers 0 and 1 remain pinned to their local core, ensuring their working memory stays resident in L1/L2 cache and eliminating cache-miss penalties.